

High Performance (Lock Free Linked List) For Parallel Computing

Priyanka Narain

Ritu Sharma

Yuvak Patel

Graduate School of Arts & Science, New York University

pn2182@nyu.edu

rs8375@nyu.edu

ygp2005@nyu.edu

Abstract

This project aims to present a high-performance implementation of a lock-free linked list tailored for parallel computing environments. In this project we aim to design the linked lists to handle concurrent access by multiple threads or processes without the need for locks. By avoiding locks via leveraging non blocking algorithms and employing atomic primitives like compare-and-swap (CAS), the implementation ensures efficient data access and manipulation across multiple threads or processes. The design prioritizes minimizing contention and maximizing throughput, resulting in improved performance in highly parallel systems. Through rigorous evaluation and experimentation, we demonstrate the efficacy of the lock-free linked list in achieving linearizable operations and facilitating seamless integration into parallel computing frameworks.

1.Introduction

One of the problems with concurrent programming is the critical section, which slows down overall performance owing to contention for shared resources that need to be accessed or modified, and for which synchronization is essential to avoid data races and assure consistency. Non-blocking algorithms are beneficial for parallel computing systems. Non-blocking algorithms avoid using locks, which means they do not make threads wait for others to release locks before they can proceed. Instead, they utilize low-level atomic operations like Compare-And-Swap (CAS).

Why a linked list ? Linked lists serve as a foundational building block for other data structures

because of their inherent flexibility and simplicity in organizing data.

A non-blocking linked list is linearizable and behaves as if all operations (like insertions and deletions) are happening instantaneously at some point between their start and end times. This property is important for maintaining consistency in concurrent operations, making the data structure predictable and reliable in a parallel computing context.

The steps involved in our approach include designing the lock-free linked list data structure, implementing algorithms for insertion, deletion, and search operations, and evaluating the performance of our solution in parallel computing environments. The study leverages the popular non-blocking Harris algorithm and CAS operation against traditional locking mechanisms.

2.Literature review

2.1 Why is lock free data structure needed at all in parallel systems ?

Traditional locking mechanisms, such as mutexes and semaphores, introduce synchronization overhead and can lead to issues like deadlock and contention, particularly in highly parallel systems.

Lock-free data structures can often achieve better scalability compared to lock-based approaches. In lock-based systems, contention for locks can increase as the number of threads or processes grows, leading to performance bottlenecks. Lock-free data structures, on the other hand, allow multiple threads

to make progress concurrently without being hindered by locks, thereby improving scalability.

2.2 Compare-And-Swap

2.2.a Basic understanding:

Compare-And-Swap, an primitive atomic operation that is widely used in concurrent programming to achieve synchronization without locks. This means the entire operation of comparing the current value, swapping it with the new value if they match, happens as a single indivisible unit. This ensures that no other thread can interfere between the check and the update, preventing data races and inconsistencies.

Success and Failure of CAS:

Success: If the current value in the memory location matches the expected value, CAS swaps the value with the new value and returns true. This indicates a successful update.

Failure: If the current value in the memory location doesn't match the expected value, the swap doesn't happen. CAS returns false, and the thread typically needs to retry the entire operation (re-read the current value, calculate a new expected value, and attempt CAS again).

2.2.b Limitations of CAS

However CAS has some limitations, CAS ensures atomicity, meaning the operation (compare and swap) happens as a single, indivisible unit. However, it doesn't guarantee that other operations won't interfere within the list structure itself.

Classic example is the race Condition where you imagine two threads, one trying to insert a new node and another trying to delete a node. Both threads might read the current state of the list (using separate CAS operations), then perform their modifications (insertion or deletion). Since CAS only checks a single value, it's possible that both threads succeed in their individual CAS operations, even though the resulting list state might be inconsistent (e.g., inserting into a deleted node).

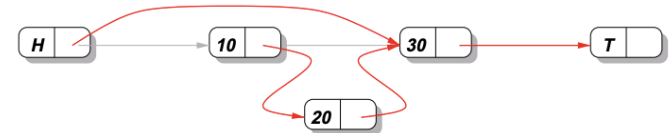
Understanding it diagrammatically:

Suppose we wish to delete the node with value 10. The simple CAS would swing the reference from the

head so that the node containing 30 becomes the first in the list.



The problem arises when deletions are concurrent with insertions. As the CAS ensures that the node 10 was still at the start of the list it cannot ensure that no additional nodes were introduced between the 10 node and the 30 node.



The CAS operation used for deletion verifies only that the node to be deleted is still at the expected location (next to the head). If another insertion occurs concurrently, like the node containing 20 being inserted before the node containing 30, the deletion CAS would not be aware of this new node. It would still swing the head's next reference to the node containing 30, effectively losing the newly inserted node containing 20.

One of the significant advancements in achieving the lock free concurrent operations on a linked list is through the Harris algorithm.

2.3 Harris algorithm

2.3.a Basic theory

The algorithm addresses the fundamental challenge of designing data structures that can be accessed and modified concurrently by multiple threads or processes without resorting to traditional locking mechanisms. The Harris algorithm offers an alternative approach by embracing the principles of non-blocking concurrency. By leveraging atomic primitives like Compare-And-Swap (CAS), the algorithm ensures that multiple threads or processes can access and modify the data structure simultaneously without the risk of deadlock or contention.

One of the key features of the Harris algorithm is its emphasis on linearizability i.e (a) the responses received in every concurrent history are equivalent to those of some legal sequential history of the same requests.

and (b) the ordering of operations within the sequential history is consistent with the real-time order.

Linearizability is a correctness criterion that ensures operations appear to take effect atomically at some point between their invocation and response. This property is crucial for maintaining consistency and predictability in concurrent operations, making the algorithm well-suited for parallel computing environments where performance and scalability are paramount.

2.3.b The Harris Algorithm: A Lock-Free Linked List Implementation

1. Data Structure Overview: The data structure consists of a singly-linked list of nodes, where each node contains a key value and a reference to the next node in the list. Additionally, the list is maintained in ascending order, with sentinel head and tail nodes to facilitate boundary handling.



2. Node State and Marking: Nodes in the linked list can be in one of two states: marked or unmarked. A marked node indicates that it is in the process of being deleted and should be ignored by subsequent operations. This marking is achieved by modifying the reference in the node's next field.

The algorithm further implements three operations ensuring the correctness criteria:

a. Search Operation (List::search): The search operation is crucial for ensuring the consistency of the list during insertions, deletions, and searches. It locates the appropriate pair of nodes (left and right) between which the operation should be performed.

The first section of the search operation iterates along the list to find the first unmarked node with a key greater than or equal to the search key. This node becomes the right node, and the left node preliminarily refers to the previous unmarked node found.

```

private Node *List::search (KeyType search_key, Node **left_node) {
    Node *left_node_next, *right_node;

    search_again:
    do {
        Node *t = head;
        Node *t_next = head.next;

        /* 1: Find left_node and right_node */
        do {
            if (!is_marked_reference(t_next)) {
                (*left_node) = t;
                left_node_next = t_next;
            }
            t = get_unmarked_reference(t_next);
            if (t == tail) break;
            t_next = t.next;
        } while (is_marked_reference(t_next) || (t.key < search_key)); /*B1*/
        right_node = t;

        /* 2: Check nodes are adjacent */
        if (left_node_next == right_node) /*C1*/
            if ((right_node != tail) && is_marked_reference(right_node.next))
                goto search_again; /*G1*/
            else
                return right_node; /*R1*/

        /* 3: Remove one or more marked nodes */
        if (CAS (&(left_node.next), left_node_next, right_node)) /*C1*/
            if ((right_node != tail) && is_marked_reference(right_node.next))
                goto search_again; /*G2*/
            else
                return right_node; /*R2*/
        } while (true); /*B2*/
    }
}
  
```

Fig 1. Search Operation

The second stage verifies that the left node is the immediate predecessor of the right node. If not, it proceeds to the third stage.

In the third stage, a CAS (Compare-And-Swap) operation is used to remove any marked nodes between the left and right nodes, ensuring that they are adjacent.

b. Insertion Operation (List::insert): The insert operation utilizes the search method to locate the pair of nodes between which the new node is to be inserted. The update itself is performed with a single CAS operation, which atomically modifies the reference in the left node's next field to point to the new node.

```

public boolean List::insert (KeyType key) {
    Node *new_node = new Node(key);
    Node *right_node, *left_node;

    do {
        right_node = search (key, &left_node);
        if ((right_node != tail) && (right_node.key == key)) /*T1*/
            return false;
        new_node.next = right_node;
        if (CAS (&(left_node.next), right_node, new_node)) /*C2*/
            return true;
        } while (true); /*B3*/
    }
}
  
```

Fig2. Insert Operation

c. Deletion Operation (List::delete): The delete operation also relies on the search method to locate

the node to be deleted.

```
public boolean List::delete (KeyType search_key) {
    Node *right_node, *right_node_next, *left_node;

    do {
        right_node = search (search_key, &left_node);
        if ((right_node == tail) || (right_node.key != search_key)) /*T1*/
            return false;
        right_node_next = right_node.next;
        if (!is_marked_reference(right_node_next))
            if (CAS (&(right_node.next), /*C3*/
                    right_node_next, get_marked_reference (right_node_next)))
                break;
    } while (true); /*B4*/
    if (!CAS (&(left_node.next), right_node, right_node_next)) /*C4*/
        right_node = search (right_node.key, &left_node);
    return true;
}
```

Fig3. Delete Operation

It is a two-stage process: Firstly, it logically deletes the node by marking the reference contained in the right node's next field.

Secondly, it physically deletes the node, either directly or through a separate invocation of the search method.

6. Find Operation (List::find):

The find operation invokes the search method and examines the resulting right node to determine the presence of the search key.

```
public boolean List::find (KeyType search_key) {
    Node *right_node, *left_node;

    right_node = search (search_key, &left_node);
    if ((right_node == tail) ||
        (right_node.key != search_key))
        return false;
    else
        return true;
}
```

Fig4. Find Operation

2.3.c. Proof of Linearizability

Checking linearizability is crucial for ensuring the correctness and consistency of concurrent algorithms, especially in scenarios where multiple threads or processes are accessing shared data structures concurrently. Linearizability guarantees that the behavior of concurrent operations on the data structure appears as if they occurred instantaneously

How to prove it ? Let's understand how linearizability is ensured in the Harris Algorithm for different operations.

- a. For the insert operation: Assume Processor A successfully inserts a node at time. The linearization point ensures that the insertion operation is completed successfully, and the CAS operation at this point guarantees that the left node is unmarked and that the right node remains its successor, preserving the ordering and consistency of the list.
- b. For the search operation, the final real-time at which the post-conditions of the search are satisfied is considered as the linearization point. Post conditions such as:
 - Presence of the Right Node
 - Ordering Constraints i.e left node < search node < right node
 - Adjacency of Nodes i.e left node is adjacent to right node.
 - Mark State of Nodes

These post conditions in the search method correctly locate the desired node or nodes within the linked list and maintain the integrity of the data structure despite concurrent modifications by other processes or threads.

- c. For the find operation the linearization point signifies that the right node was unmarked and contained the search key.
- d. For delete operations, Firstly, it verifies that the right node is still unmarked immediately before the update, ensuring that it has not been logically deleted by a preceding successful deletion. Secondly, the update itself marks the right node, making the deletion visible to other processors and maintaining the integrity of

2.3.d. Proof of non blocking implementation:

While linearizability gives an impression that the operations are executed one after the another in a sequential manner, even though they are executed concurrently. Progress on the other hand, ensures that the operations can complete successfully, even in the presence of contention or concurrent modifications by other threads.

How to prove it ? Let's understand how progress is ensured in the Harris Algorithm for different operations.

- a. For insert operation: (In fig2 : insert operation) Each successful insertion involves exactly one update, represented by the CAS operation C2. C2 succeeds by swinging the reference in the left node's next pointer to point to the new node, effectively inserting it into the list. This update ensures that the newly inserted node is properly linked into the list, preserving the integrity of the data structure. Unsuccessful insertions do not cause any updates, as the CAS operation C2 fails to complete due to contention with other threads.
- b. For delete operation: (In fig3 : delete operation) Successful deletions may involve up to two updates, represented by CAS operations C3 and C4. C3 logically deletes the node by marking its next pointer, while CAS operation C4 physically removes the node from the list. However, due to the marking of the node in CAS operation C3, only one of C3 or C4 can succeed in each successful deletion operation, ensuring that at most two updates occur. Unsuccessful deletions do not cause any updates, as the CAS operations C3 and C4 fail to complete due to contention with other threads.

2.3.e Pros and Cons of Harris algorithm

While non blocking concurrency , linearizability , scalability and high performance are the pros of the algorithm. There are certainly some limitations as well:

1. Dangling memory location : When nodes are removed from the data structure, it's essential to ensure that memory allocated to those nodes is properly reclaimed to prevent memory leaks. Techniques such as hazard pointers can be used to address memory reclamation however, they add complexity and overhead to the data structure's design and implementation.

2. Re-traversing the List: In scenarios where a CAS failure occurs during an insertion or deletion operation, the thread may need to re-traverse the list to locate the appropriate nodes for performing the

operation. This re-traversal ensures that the thread operates on the most up-to-date state of the linked list, taking into account any concurrent modifications by other threads.

2.4 Proposed Idea

As we discussed earlier, with each failed CAS the thread needs to re-traverse the entire linked list which adds latency and drops the performance in concurrent operation. One of the solutions to mitigate this could be partial re-traversal of the linked list starting from the last known position or utilizing checkpoints within the list structure.

How can the checkpoints be determined ?

Temporal locality: Nodes that are accessed or modified recently are more likely to be accessed again soon. This principle can be leveraged to dynamically determine hotspots by marking nodes accessed during operations and using them as starting points or checkpoints for future operations.

Spatial locality: If operations on a node often lead to operations on its adjacent nodes, this suggests a clustering effect. For example, in a list sorted by some criterion, operations on a particular range of values might cluster around specific nodes that serve as entry points to these values.

We have implemented the lock free linked list that considers both the full retraversal and partial traversal to understand its impact on performance.

In the full retraversal , all the three operations insert, delete, and search always start from the head of the list. When a CAS operation fails due to contention, the entire operation restarts from the head, traversing potentially the entire list again to locate the appropriate node, which can be inefficient, especially for large lists or high contention scenarios.

While in the modified version, we propose to start the search for the appropriate position from the checkpoint and not always from the head.

Understanding it with an example:

Partial Traversal (Modified Code with Checkpoint)

Head -> 10 -> 20 -> 30 -> 40 -> 50 -> Tail

Let's assume that Thread A wants to insert 35 into the list and Thread B wants to delete 30 from the list. The latest checkpoint is at 30.

Step 1: Thread A Attempts to Insert 35. Thread A starts from the checkpoint at 30. It moves forward and finds that 35 should be placed between 30 and 40. As it prepares to insert, it sets up its local variables and conditions for a CAS operation.

Step 2: Thread B Attempts to Delete 30. Thread B also starts from the checkpoint at 30. It confirms the next node is 40 and prepares to perform a CAS to unlink 30, setting 20's next pointer to 40.

Step 3: Concurrent Execution Dynamics : Suppose Thread B successfully unlinks 30 right before Thread A executes its insertion CAS. Thread A's CAS operation fails because its expected next pointer of 30 is no longer valid (the structure has changed due to the deletion by Thread B).

Step 4: Thread A's Partial Re-traversal Thread A, upon failing the CAS, does not restart from the head. Instead, it uses its last known good position (right at 30 or just before 30 at 20 if it detects the change). It now sees that 30 is gone and 20 links directly to 40. It adjusts to insert 35 between 20 and 40. Thread A completes the insertion successfully.

Step 5: Checkpoint Update After Thread A inserts 35, it updates the checkpoint to 35 if this node is deemed to likely be a future hotspot (e.g., due to its value or position). Similarly, Thread B after deleting 30, might adjust the checkpoint back to 20, anticipating further operations might focus around this newly adjusted area.

```
bool Search(const T& data, Node** prev_ptr, Node** cur_ptr) {
    Node* prev = checkpoint_load(std::memory_order_acquire);
    Node* cur = prev->next.load(std::memory_order_acquire);

    while (cur != nullptr) {
        if (cur->data == data) {
            *prev_ptr = prev;
            *cur_ptr = cur;
            if (cur->data == data) {
                checkpoint_store(cur, std::memory_order_release); // Update checkpoint
                return true;
            }
        }
        return false;
    }
    prev = cur;
    cur = cur->next.load(std::memory_order_acquire);
}

*prev_ptr = prev;
*cur_ptr = cur;
return false;
}
```

Fig5 : Update Checkpoint

In our simplified version of checkpoint, the initial checkpoint is Head while for subsequent searches it updates the checkpoint to the current node.

There are scenarios where the checkpoint can be proven to be beneficial, one of which is:

Scenario: A real-time collaborative application where users are continuously updating a shared document or data structure. In such a scenario, starting from the head for every operation could be highly inefficient, especially if the operations are concentrated around certain areas far from the head. So by applying dynamic checkpointing to the frequently accessed regions, the list traversal becomes much faster.

Experimental analysis of checkpoints:

With using checkpoint (current node pointer in our case) we have observed significant change in the performance of concurrent insertions and deletions.

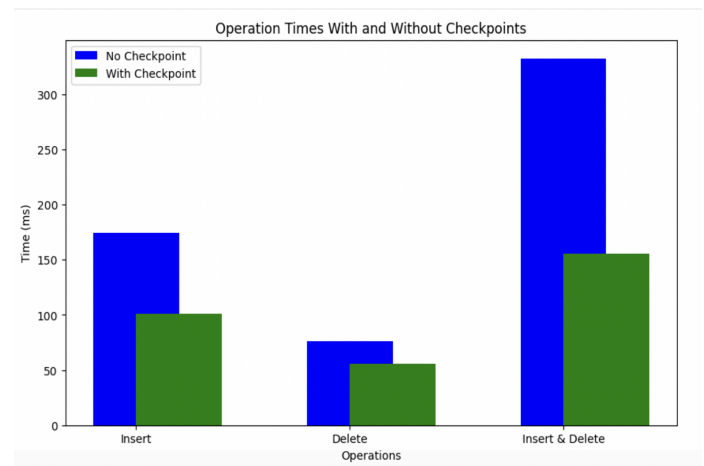


Fig 6 : Checkpoint results

Above observations demonstrate that checkpointing reduces latency and increased efficiency, particularly in environments with frequent concurrent updates as against the full retraversal of the linked list with every failed CAS.

2.5 Experimental setup :

We have conducted an experiment in CIMS crunchy with 64 logical cores. Here's how we have setup the experimental :

- Fixed Problem Size Element - 1000, 10000:**
The experiment was conducted with two fixed problem sizes: 1000 elements and

10000 elements. For each problem size, the number of threads varied from 64, 32, 16, 8, 4, 2, 1. This setup allowed for testing the performance of the algorithm under varying degrees of parallelism, ranging from the maximum hardware concurrency down to a single thread.

- b) No of Threads as 64 (Hardware Concurrency): The experiment focused on keeping the number of threads constant at the maximum hardware concurrency, which was 64. Three different problem sizes were tested: 100, 1000, and 10000 elements. This setup aimed to observe how the algorithm performed when fully utilizing the available hardware threads for different problem sizes.
- c) No of Threads as 32: Similarly, the experiment maintained a constant number of threads at 32. Three problem sizes were tested: 100, 1000, and 10000 elements. This setup allowed for evaluating the algorithm's performance with a moderate number of threads across different problem sizes.
- d) No of Threads as 16, 8: The experiment continued by keeping the number of threads constant at 16 and 8, respectively. Similar to previous setups, tests were conducted with problem sizes of 100, 1000, and 10000 elements. These setups aimed to explore the algorithm's behavior with fewer threads while varying the problem size.

We will be evaluating the performance of our experiment based on both full & partial retraversals and the following parameters and:

1. Throughput
2. SpeedUp
3. Efficiency
4. CPU time

2.6 Experiments and Analysis:

Below are the experimental results based on the number of threads and problem size.

I. Performance Metrics vs. Number of threads

For these operations, the time taken generally decreases as the number of threads increases from 1 to 8. This is because additional threads allow work to be done in parallel, increasing the throughput of the system. However, after a certain point, the time taken

starts to increase as the number of threads goes beyond 16. This could be due to several factors such as overhead, thread contention etc.

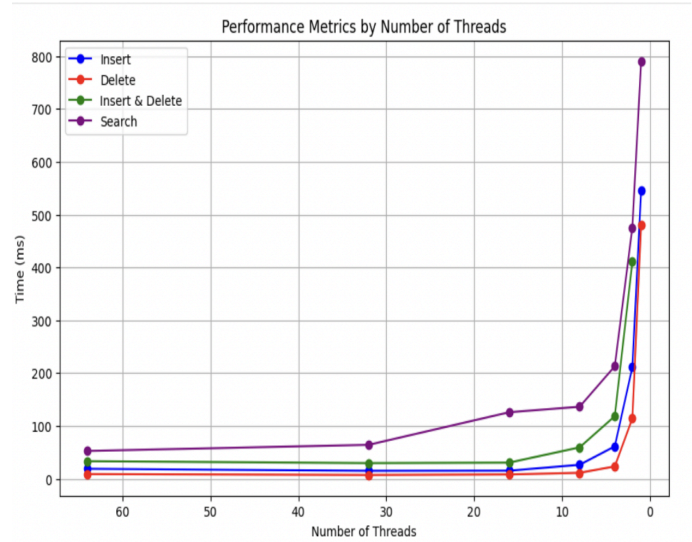


Fig 7 : Performance metrics on 100000 elements

II. Throughput and Speedup Metrics

The throughput graph for 1,000 elements illustrates the reality of overhead in multi-threaded operations. Particularly, it shows that for this problem size, employing 32 or 64 threads does not necessarily enhance performance; in fact, it introduces overhead. This is a classic example of diminishing returns, where the cost of managing additional threads outweighs their benefit, underscoring the importance of tuning the thread count to the specific workload.

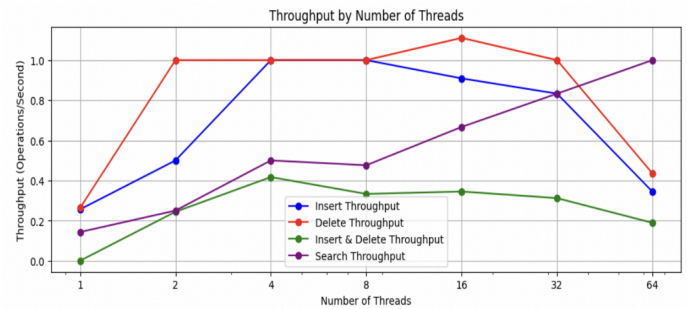


Fig 8 : Throughput metrics on 1000 elements

As the Fig 8 displays a clear trend of increasing throughput as the number of threads rises, which indicates that the workload is sufficiently large to benefit from higher levels of concurrency. With more threads available, the workload can be distributed more effectively, leading to increased parallel processing and better utilization of system resources.

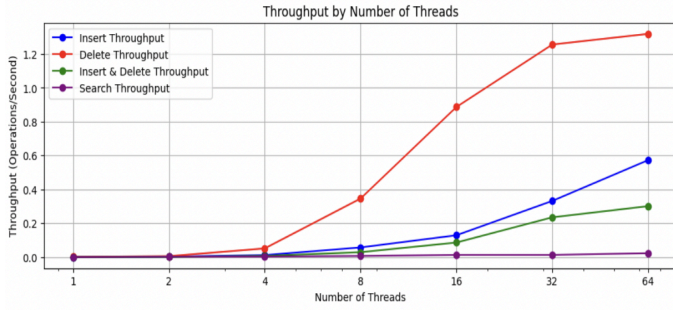


Fig 9 : Throughput metrics on 100000 elements

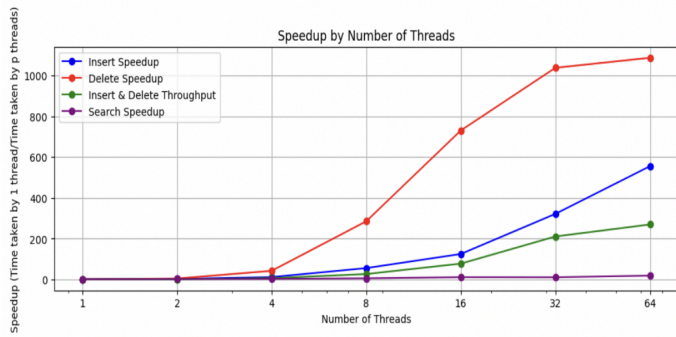


Fig 10 : Speedup metrics on 100000 elements

III. Efficiency metrics

The efficiency graph depicted reflects the scaling of parallelization with respect to thread count, particularly highlighting that delete operations initially capitalize on parallel processing, as evidenced by the surge in efficiency up to 8 threads. Beyond this point, efficiency diminishes as threads increase, likely due to contention for Compare-And-Swap (CAS) operations and the critical sections during deletion. With more threads in the mix, the probability escalates that numerous threads will be stalled, awaiting the completion of these atomic operations, leading to a bottleneck which in turn hampers overall efficiency.

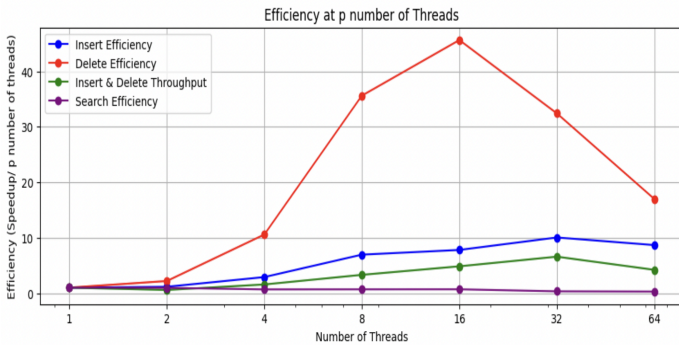


Fig 11 : Efficiency metrics on 100000 elements

IV. CPU Time metrics

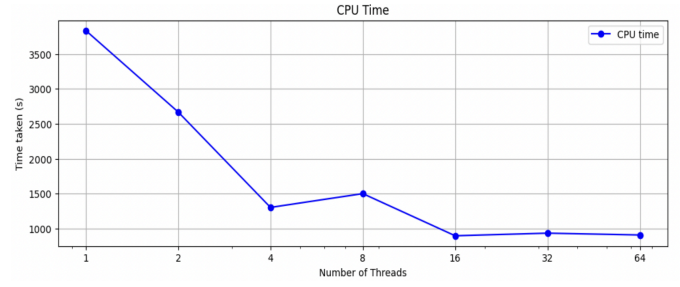


Fig 12 : CPU Time metrics on 100000 elements

The CPU time plot for a problem size of 100,000 exhibits a declining trend as the number of threads increases from 1 to 16, indicating effective parallelization. Beyond 16 threads, CPU time plateaus and even increases slightly with 64 threads, suggesting that additional threads beyond a certain threshold do not contribute to performance gains, likely due to the overhead from managing a higher number of threads and contention among them.

The comparative results strongly suggest that lock-free linked lists excel over sequential ones, particularly when managing a substantial number of elements. The lock-free implementation leverages parallelism effectively, evidenced by higher throughput with increased threads. This contrast is especially pronounced at larger scales, where lock contention significantly hampers sequential linked lists, confirming the superiority of lock-free structures in high-concurrency scenarios.

2.7 Conclusion :

1. **Enhanced Scalability:** The lock-free design scaled effectively with increasing threads, showing substantial gains in throughput and speedup in high-concurrency settings.
2. **Efficient Data Handling:** Utilization of non-blocking algorithms i.e the Harris algorithm and atomic operations (CAS) significantly reduced the overhead and contention compared to traditional lock-based mechanisms, enhancing overall efficiency, especially in deletion and insertion operations.
3. **Advantages of Dynamic Checkpointing:** Based on the proposed idea implementing dynamic checkpointing within the linked list to manage partial traversal effectively in our case mitigated the performance drop typically

associated with full retraversals after CAS operation failures. In addition, further exploration on checkpointing based on temporal and spatial locality, could lead to faster operations and reduced latency in environments with frequent and concurrent modifications.

2.8. References

1. <https://www.cl.cam.ac.uk/research/srg/netos/papers/2001-caslists.pdf> A Pragmatic Implementation of Non-Blocking Linked-Lists by Timothy L. Harris
2. <https://www.youtube.com/watch?v=WYJBdAuX7zw>
3. <https://www.cs.otago.ac.nz/cosc440/readings/hazard-pointers.pdf> Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects by Maged M. Michael
4. <https://github.com/bhhbazinga/LockFreeLinkedList/>
5. <https://www.cse.wustl.edu/~angelee/archive/cse539/spr15/lectures/lists.pdf>